CS 327: Compilers
Language Specification for **CYLite**

Swayam Borate (23110066) | Shardul Junagade (23110297) | Tejas Lohia (23110335)

06 February 2026

1 Introduction

This PDF is the language rulebook for **CYLite**. It defines what our compiler will support across assignments A2–A4 (lexer, parser, and semantic analysis). The intent is to keep the language close to a small, C-like core so that it remains implementable using `lex` and `yacc` in C.

Reference baseline. We used the provided *YAPL* language specification as a starting reference. **CYLite** follows the same overall C-like style (expressions, statements, declarations), but the set of supported constructs is chosen by us and includes a few small extensions listed in Section 6.

2 Lexical specification

2.1 Whitespace

Spaces, tabs, and newlines may appear between tokens. They are ignored by the lexer, except that newlines end single-line comments.

2.2 Comments

- **Single-line (C++ style):** `//` continues up to end of line.
- **Single-line (extension):** `#` continues up to end of line.
- **Multi-line:** `/*` begins a comment that ends at the next `*/`.

If the input ends before a multi-line comment is closed, the lexer must report an ill-formed (unclosed) comment.

2.3 Identifiers

Identifiers follow the usual C rule.

```
[A-Za-z_][A-Za-z0-9_]*
```

2.4 Keywords

The following words are reserved and cannot be used as identifiers.

Core keywords (C-like).

`int`	`long`	`short`	`char`
`float`	`double`	`void`	`struct`
`extern`	`return`	`break`	`continue`
`if`	`else`	`switch`	`case`
`default`	`for`	`while`	`do`

Keywords added in CYLite.

`elif`	`pass`	`try`	`except`
`print`	`range`	`in`	`foreach`

Any standard C keyword not listed above is treated as a normal identifier in **CYLite**.

2.5 Literals

Integer literals. We support decimal integers and hexadecimal integers of the form `0x...`.

- decimal: `0` or `[1-9][0-9]*`
- hexadecimal: `0x[0-9a-fA-F]+`

Floating-point literals. We support decimal floats such as `3.14`, `0.5`, and `10.`.

Character literals. Single-quoted characters like `'a'` are supported. Basic escape sequences may be supported (e.g., `\n`, `\t`, `\'`, `\\`).

String literals. Double-quoted strings like `"hello"` are supported. Adjacent strings are concatenated (C-style): `"a" "b"`.

2.6 Punctuation

The following punctuation tokens are used in the grammar: `( )`, `[ ]`, `{ }`, `,`, `;`, `:`.

3 Data types

3.1 Primitive types

CYLite supports the following built-in types:

- `int`, `long`, `short`, `char`
- `float`, `double`
- `void`

3.2 Derived types

- **Pointers:** any type may be pointer-qualified, e.g., `int *****p;`.
- **Arrays:** any non-function type may be an array type, e.g., `int a[10];`.
- **Structs:** user-defined aggregate types via `struct`.
- **Functions:** functions may return any non-array type (including pointers).

4 Operators and precedence

Unless stated otherwise, operator precedence and associativity follow C for the supported operators.

Special operators. `sizeof` is treated as an operator token (not a reserved keyword). Supported forms are `sizeof(type)` and `sizeof(unary_expr)`. Type casts of the form `(type)expr` are supported.

Category	Tokens
Arithmetic	`+ - * / %`
Relational	`< > <= >= == != <=>`
Logical	`&& || !`
Bitwise (subset)	`& | ^ ~`
Assignment	`=`
Inc/Dec	`++ -`
Pointer / address	`* &`
Field access	`. ->`
Calls / indexing	`( ) [ ]`
Special	`sizeof`

Table 1: Operators in **CYLite**

Not supported. We do not support preprocessors/directives, ternary `?:`, compound assignments (`+=`, `-=`, ...), shift operators («, »), or varargs/ellipsis (`...`).

5 Language constructs (overview)

Declarations and definitions

- Global variables (including `extern` declarations).
- Function declarations (prototypes) and function definitions (with bodies), including recursion.
- Local declarations inside blocks; multiple declarators separated by commas.
- `struct` types and field access using `.` and `->`.

Statements

- Selection: `if` / `else` / `elif` ladders; `switch` / `case` with optional `default` and fall-through.
- Iteration: `for`, `while`, `do-while`, restricted `for (i in range(l, r))`, and `foreach`.
- Jump: `break`, `continue`, `return`.
- Empty/no-op: `pass;` and the empty statement `;`.

6 Extensions over the YAPL baseline

The following are additions that are specific to **CYLite**.

- `elif`: Supported as syntactic sugar for `else if`. It can appear zero or more times between an `if` and a final optional `else`, keeping if-ladders readable without forcing deeper nesting.
- `pass;`: An explicit no-op statement. It is equivalent to writing an empty statement `;`, but it makes intent clear in places like empty loop bodies or placeholder branches.

- **`print(...)`:** The call syntax is the usual function-call form with parentheses, and we allow a comma-separated argument list (including a single argument).
- **`range(l, r)`:** A built-in that represents the integer sequence from `l` (inclusive) to `r` (exclusive). In **CYLite**, it is primarily used by the restricted range-loop form.
- **Restricted range loop:** A restricted Python-style loop form:

  ```
  for (i in range(l, r)) stmt
  ```

  Restrictions: the loop variable must be an identifier, and the iterator must be exactly `range(l, r)` with two expressions. Informally it behaves like iterating `i = l` while `i < r`.
- **`#` comments:** In addition to `//`, `#` starts a single-line comment. Since preprocessors are not part of **CYLite**, lines starting with `#` are always treated as comments and not as directives.
- **Try/except:** Python-like exception handling using block syntax:

  ```
  try { ... }
  except { ... }
  ```

  Only a single catch-all `except` is supported. We do not require exception types, matching, or raising in the current version.
- **Foreach loop:** `foreach` iterates over arrays and over string literals. Each iteration assigns the current element to the loop variable.
  Foreach syntax:

  ```
  foreach(x in array) { ... }
  foreach(c in "hello") { ... }
  ```

- **Optional operators (only if integration stays clean):** These will be included only if precedence/associativity handling in `yacc` remains clean.
  - `**` exponent operator (intended right-associative and higher precedence than `* / %`).
  - `??` null-coalescing operator (intended right-associative).

7 Grammar / syntax rules

This is a compact, lexer-friendly grammar sketch. It is not meant to be a complete C grammar, but it defines the constructs we plan to accept.

```
program        := external_decl*

external_decl  := decl ';'
                | func_proto ';'
                | func_def

func_proto     := type ptr* id '(' param_list? ')'
func_def       := type ptr* id '(' param_list? ')' compound_stmt

decl           := type decl_item (',' decl_item)*
decl_item      := ptr* id array_suffix* ('=' expr)?
array_suffix   := '[' expr? ']'

type           := prim_type | 'struct' id
prim_type      := 'int' | 'long' | 'short' | 'char' | 'float' | 'double' | 'void'
ptr            := '*'
```

```
param_list     := param (',' param)*
param          := type ptr* id array_suffix*

compound_stmt  := '{' decl_or_stmt* '}'
decl_or_stmt   := decl ';' | stmt

stmt           := compound_stmt
                | expr ';'
                | ';'
                | 'pass' ';'
                | if_stmt
                | while_stmt
                | do_while_stmt
                | for_stmt
                | range_for
                | foreach_stmt
                | switch_stmt
                | try_except
                | jump_stmt

if_stmt        := 'if' '(' expr ')' stmt
                                ( 'elif' '(' expr ')' stmt )*
                                ( 'else' stmt )?

while_stmt     := 'while' '(' expr ')' stmt
do_while_stmt  := 'do' stmt 'while' '(' expr ')' ';'
for_stmt       := 'for' '(' expr? ';' expr? ';' expr? ')' stmt
range_for      := 'for' '(' id 'in' 'range' '(' expr ',' expr ')' ')' stmt

foreach_stmt   := 'foreach' '(' id 'in' expr ')' compound_stmt

switch_stmt    := 'switch' '(' expr ')' '{'
                                ( 'case' const ':' stmt* )*
                                ( 'default' ':' stmt* )?
                                '}'

try_except     := 'try' compound_stmt 'except' compound_stmt

jump_stmt      := 'break' ';'
                | 'continue' ';'
                | 'return' expr? ';'

const          := int_lit | char_lit
```

8 Semantic rules (planned for A4)

This section states the semantic rules we intend to enforce in the semantic analysis phase. For A2 (lexer) and A3 (parser), these are documented here but do not need to be fully enforced.

Scoping

- Block scoping: each `{...}` introduces a new scope.
- Shadowing is allowed: an inner declaration may reuse an outer name.
- Function parameters are in the function's outermost local scope.

Declarations and usage

- Variables and functions must be declared before use within the same scope (C-like rule).
- A function prototype and a function definition may both appear; the definition must match the declared signature.

Type checking (high level)

- Arithmetic operators apply to numeric types; relational operators produce an integer truth value.
- Assignment requires compatible types (with standard numeric promotions).
- Pointer operations: `&` yields a pointer, `*` dereferences a pointer.
- Field access: `.` requires a struct value, `->` requires a pointer-to-struct.
- `sizeof` returns an integer type.

Foreach typing

- For `foreach(x in array)`, the loop variable `x` must be assignable from the element type of `array`.
- For `foreach(c in "...")`, `c` is treated as a `char`.

9 Example programs

The following examples are small and designed to exercise the supported tokens and constructs.

Example 1: if/elif/else and print

```
int main() {
        int x = 5;
        if (x < 0) {
                print("neg");
        } elif (x == 0) {
                print("zero");
        } else {
                print("pos");
        }
        return 0;
}
```

Example 2: range-based loop

```
int main() {
        int i;
        for (i in range(0, 5)) {
                print(i);
        }
        return 0;
}
```

Example 3: foreach over an array and a string

```
int main() {
        int a[3];
        a[0] = 10; a[1] = 20; a[2] = 30;

        int x;
        foreach(x in a) {
                print(x);
        }

        char c;
        foreach(c in "hi") {
                print(c);
        }
        return 0;
}
```

Example 4: try/except and pass

```
int main() {
        try {
                pass;
                print("inside try");
        } except {
                print("inside except");
        }
        return 0;
}
```

10 Restrictions and design choices

- Statements such as expression statements and jumps end with `;`. Compound statements use braces.
- We aim to keep the grammar small enough to be implementable cleanly in `lex`/`yacc`.